

Traj-VLN: Learning Pixel-Space Interaction via Autoregressive Trajectory Generation

Changfei Fu, Guangcheng Chen, Aoxiang Gu, Haoxiang Liang,
Wenjun Xu[†], and Hong Zhang[†], *Life Fellow, IEEE*

Abstract—Benefiting from the powerful priors embedded in large-scale pre-training data and the emerging commonsense reasoning ability, large language models (LLMs) have shown unprecedented generalization capabilities in many research fields. Recently, projecting visual embeddings into the language space via vision-language models (VLMs) to achieve sim-to-real and cross-scene generalization has become a prevailing paradigm in the field of Vision-and-Language Navigation in Continuous Environments (VLN-CE). VLN requires an embodied agent to navigate through unseen environments following natural linguistic instructions. We emphasize that a VLN task can be decomposed into a sequence of sub-tasks, each corresponding to a process of 3D spatial interaction with the environments described by instructions such as “walk to the end of the sofa and turn left.” However, such spatial interactions involving moving into the image along the direction of depth sensing are puzzling for VLMs as they were predominantly trained on conversations with RGB images.

Rather than incorporating depth or 3D geometric information-which VLMs rarely encounter during pretraining—we propose an alternative approach: fine-tuning VLMs to learn navigation interactions directly in 2D pixel space through autoregressive trajectory generation. Given a linguistic instruction and historical observations, our model sequentially predicts a series of pixel coordinates, drawing a trajectory from the bottom center of the current observation. While prior work has proved that pixel-goal supervision outperforms learning of discrete actions, our experiments further verify that the supervision of pixel-space trajectory significantly enhances VLN performance. Moreover, we demonstrate that our flagship model achieves state-of-the-art level performance with relatively limited computational resources and training data.

I. INTRODUCTION

Vision-and-language navigation (VLN) requires an embodied agent to navigate in unseen environments following natural language instructions, while receiving continuous visual observations from onboard sensors. The instructions are typically composed of detailed sub-task descriptions—for example, “walk out of the dining room, into the living room, and take the first right into the recreation room; stop between the door and the pool table.” Unlike traditional navigation paradigms, which focus on moving the agent from one

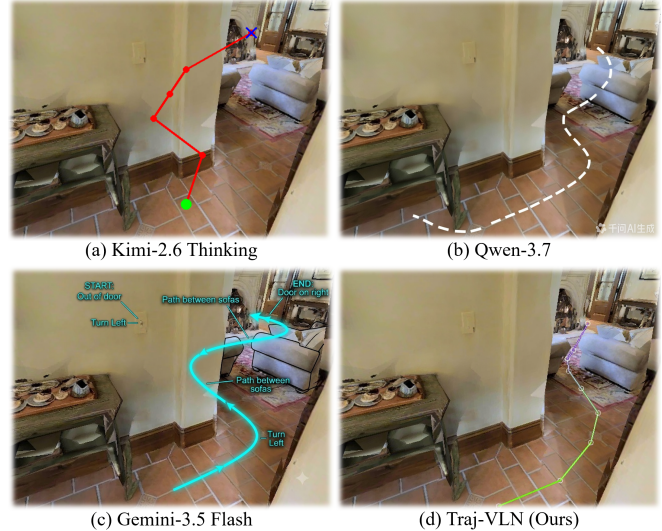


Fig. 1: Qualitative comparison of Traj-VLN with general-purpose multi-model large language models (MLLMs). It’s demonstrated that the popular MLLMs can hardly draw a reasonable trajectory to interact with the 3D environments. With our fine-tuning supervised by pixel-space trajectories, the VLMs achieve the interactions in 2D image plane by learning to autoregressively generate a sequence of pixel coordinates. The prompt used for the general-purpose MLLMs is “please draw a trajectory on the image to follow the instruction: get out of the door and turn left, go through the way between the sofas, pass the fireplace to reach the door on the right.”

position coordinate to another, VLN emphasizes the process of 3D spatial interactions with the environment [1]. Driven by the powerful generalization capabilities of LLMs, employing VLMs to project visual information into language space has become a prevailing paradigm in VLN methods [2]–[4].

Existing VLN models mostly fine-tune VLMs supervised by direct action chunks [1], [2], [4]–[6], which in fact compose a movement trajectory in the 3D environment. However, existing VLMs and their vision encoders almost exclusively inherit the CLIP paradigm pre-trained on conversations with 2D images. Planning a 3D trajectory is inherently difficult without perceiving the 3D physical layout (Fig. 1). Recent advances have attempted to incorporate depth information [1] or geometric priors [7] into the VLN framework, yet these approaches have not completely resolved the problem, as VLMs lack sufficient exposure to 3D information utilization during pretraining [8], [9]. Consequently, additional learnable encoders are often incorporated to align depth embeddings with RGB representations [7], [9].

To formulate VLN as a pixel-grounding task compatible with pre-trained VLMs, InternVLA-N1 [11] first fine-tunes the VLM using pixel-goal supervision within the 2D

[†]Corresponding author (hzhang@sustech.edu.cn)

Changfei Fu and Hong Zhang are with the Shenzhen Key Laboratory of Robotics and Computer Vision, Southern University of Science and Technology, Shenzhen, China. Changfei Fu and Wenjun Xu are also with the Peng Cheng National Laboratory, Shenzhen, China. Weinan Chen is with the State Key Laboratory of Precision Electronic Manufacturing Technology and Equipment, Guangdong University of Technology, Guangzhou, China. This work was supported by the Shenzhen Key Laboratory of Robotics and Computer Vision (ZDSYS20220330160557001), the Major Key Project of PCL (PCL2024A04), and the National Natural Science Foundation of China under Grant U21A20476.

pixel space that is inherently familiar to VLMs. Through controlled single-variable experiments, Goal2Pixel [12] further demonstrates that pixel-goal supervision substantially improves VLN performance over action-chunk prediction.

The distribution mismatch between the large-scale internet pretraining corpus of VLMs and the robotics fine-tuning data can lead to catastrophic forgetting [10], which may explain the remarkable success of pixel-goal supervision [3], [11], [12]. However, a critical question arises: how can VLMs accurately infer a target pixel on the image if they lack explicit understanding of the 3D scene layout? We argue that the objective of VLM fine-tuning should be to reorganize and leverage the priors embedded in pre-training data, alongside the emerging reasoning capabilities. Assuming that learning pixel-space interaction processes over image regions can enhance VLN performance, we propose to fine-tune VLMs using 2D trajectories paired with detailed textual descriptions of the corresponding interactions. This paired supervision enables the model to learn the unambiguous 2D projections of 3D trajectories within the familiar 2D pixel space, circumventing the need for explicit 3D reasoning. Motivated by enabling the model to perform such interactions via drawing a trajectory from near to far, we fine-tune VLMs to autoregressively generate a sequence of pixel coordinates using a simple cross-entropy loss. The textual format of these coordinates aligns naturally with the pretraining corpus, ensuring the model understands the generated representations. The autoregressive mechanism, where the previous predictions serve as input for subsequent generation, drives the VLM to capture the sequential nature of the interaction processes, effectively functioning as a chain-of-thought (CoT) [27] for pixel-goal prediction.

In this paper, we propose Traj-VLN, which enables VLMs to learn pixel-space interactions and thereby achieves superior target pixel prediction. For each sequence of observations in the training data, we project future agent poses onto the current observation to obtain the ground-truth subsequent trajectory for supervision. Occluded and duplicate waypoints are filtered out using the depth image. The contributions of this paper are summarized as follows: 1) We propose Traj-VLN, which fine-tunes general-purpose VLMs using paired supervision from 2D trajectories and language instructions, enabling the model to autoregressively generate the trajectory waypoints in the pixel space and thereby learn the underlying interaction processes. 2) Our experiments demonstrate that the VLN model supervised by 2D trajectories significantly outperforms its counterpart supervised by pixel-goal under identical settings. 3) By adopting the final point of the generated trajectory as the pixel-goal for evaluation, we reveal that autoregressive trajectory generation effectively serves as a CoT mechanism for better pixel-goal prediction.

II. RELATED WORK

Our Traj-VLN address the spatial reasoning bottleneck in VLM-based VLN-CE by introducing 2D trajectories as the supervision signals to fine-tune VLMs in the pixel space,

which is inherently aligned with the distribution of pre-training data.

A. VLN-CE by Fine-tuning the Pretrained VLMs

VLN has recently witnessed remarkable progress in simulation and benchmarking from nodes selecting in the discrete-defined nav-graph [14]–[17] to more realistic VLN in continuous environments (VLN-CE), where the robot is allowed to navigate to any unobstructed locations [18], [19]. Further, to address the limitations in sim-to-real and cross-scene generalization caused by artificially customized neural networks and scarce training data [20], [21], recent approaches have introduced general-purpose VLMs into this research field. Leveraging the powerful priors from large-scale pre-training data and the emerging commonsense reasoning capability of LLMs, VLM fine-tuning [1]–[6], [11] has become a prevailing paradigm in VLN-CE.

As the first endeavour of VLM fine-tuning for VLN-CE, NaVid [2] inherits the main architecture of LLaMA-VID [22], using four visual tokens and one instruction-queried visual token to represent historical observations. However, information redundancy is inevitable across consecutive visual observations [6]. We adopt an uniform down-sampling strategy for historical observations, which has been widely validated as an effective practice [1], [3], [4], [11], [12].

Most existing VLN models [1], [2], [4]–[6] formulate the task as a Partially Observable Markov Decision Process (POMDP), where the VLM directly predict the next action based on the current observation, and the agent executes the predicted action to obtain a new observation. Navid-4D [7] further demonstrated that an action chunk of capacity 4 achieve the best performance. Such an action chunk, composed of primitive actions such as "move forward 25cm" and "turn left/right 15 degrees", constructs a trajectory in 3D space. However, VLMs and their vision encoders are predominantly pre-trained on conversations involving only 2D images, as datasets containing 3D geometry or RGB-D images remain scarce. To bridge this gap, NaVid-4D and JanusVLN incorporated RGB-D images and 3D priors in geometric foundation models into their VLN frameworks, yet achieved limited improvements through aligning the 3D information with the pre-trained 2D space [1], [7]–[9]. By introducing pixel-goal as supervision for VLM fine-tuning, InternVLA-N1 [3] outperforms these models that incorporate 3D information, demonstrating that effective 2D-space supervision can surpass explicit 3D alignment.

B. Supervision Paradigms for VLMs in VLN-CE

Currently, most VLN models supervise the VLMs by low-level robot actions during fine-tuning [1], [2], [4]–[6]. While InternVLA-N1 [11] pioneered the practice of pixel-goal supervision, Goal2Pixel [12] further proved through single-variable experiments that pixel-goal supervision outperforms low-level action prediction.

To establish the trajectory supervision paradigm in the pixel space, our motivations are threefold: (1) A pixel trajectory serves as a unique projection of the 3D trajectory, inherently capturing the spatial interaction process described by

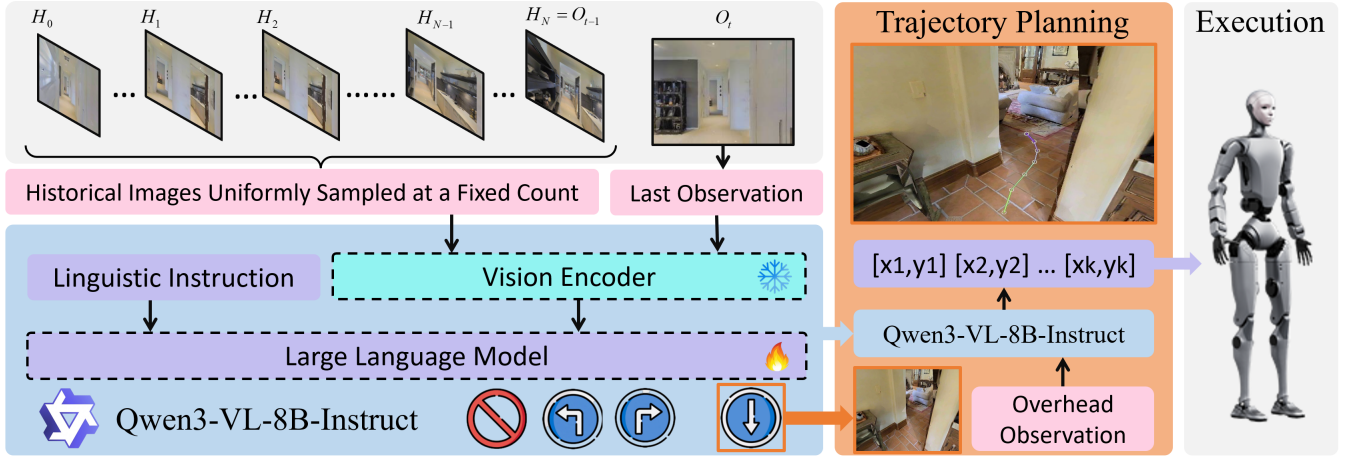


Fig. 2: The framework of Traj-VLN. At each time step, Traj-VLN takes as input a language instruction, the current observation, and a set of uniformly sampled historical observations. Based on Qwen3-VL, Traj-VLN fully fine-tunes the LLM and the projection layer while keeping the vision encoder frozen. In the first conversation stage, Traj-VLN is fine-tuned to output either turning actions or “STOP”. If Traj-VLN outputs “↓” in the first stage, it enters the second stage and receive an overhead observation. During the second stage, Traj-VLN is fine-tuned to autoregressively generate a sequence of pixel coordinates on the overhead image. Supervised by the pixel-space trajectories, Traj-VLN achieves interactions with the image regions by delineating a trajectory from near to far.

the instruction. (2) Learning the 2D interactions in the pixel space inherently matches the pre-training data distribution. (3) Recognizing the spatial reasoning limitations of existing VLMs, we hypothesize that learning to autoregressively generate a trajectory can function as a chain-of-thought (CoT) mechanism for pixel-goal prediction, enabling the model to decompose the final goal into sequential intermediate steps.

III. METHOD: TRAJ-VLN

A. Task Formulation

Vision-and-Language Navigation in Continuous Environments (VLN-CE) requires an agent equipped with onboard sensors to navigate through unseen environments by following natural language instructions. These instructions typically consist of detailed descriptions of the navigation process, e.g., “Walk out of the dining room, into the living room, and take the first right into the recreation room. Stop between the door and the pool table.”

At time step t , in addition to the instruction, the agent receives the current RGB observation O_t alongside the past observations $O_p = \{O_1, O_2, \dots, O_{t-1}\}$. To reduce computational complexity, O_p is often compressed via down-sampling [1], [11], [13] or token merging [2], [6]. Given these inputs, VLN models are expected to output language planning [24], pixel-level goals [11], [12], or even direct action chunks [7]. Consequently, a VLN model must comprehend the instruction, reason over historical observations to infer the remaining subtasks, and accurately predict the subsequent output conditioned on the current observation.

B. Model Paradigm

As shown in Fig. 2, we build Traj-VLN upon Qwen3-VL-8B-Instruct, a publicly available, general-purpose VLM. In all implementations, we keep the vision encoder frozen and fine-tune the LLM and the projection layer. We uniformly sample the historical observations $O_{his} =$

$\{H_1, H_2, \dots, H_N\}$ from O_p at a fixed count with dynamically adjusted intervals. Each image in O_{his} is resized to a smaller resolution (usually 392×392), while the current observation O_t is maintained at 640×480 . Prior to model input, each image is further rounded so that its height and width are multiples of 28, complying with the merged patch size requirements of Qwen3-VL.

We employ a chat template to constrain the VLM’s output to either “STOP” or one of the three directional symbols: “←”, “→”, or “↓”. If the agent has reached the target position when the VLM outputs “STOP”, the task is declared succeeded; otherwise failed. If the VLM outputs a sequence of “←” or “→”, the current conversation is restarted with updated vision input. If the VLM outputs “↓”, the conversation continues with an additional overhead observation incorporated as input. According to all the visual inputs, particularly the overhead image at the current position, the VLM is trained to autoregressively generate a sequence of normalized pixel coordinates formatted as “[u_1, v_1] [u_2, v_2] ... [u_k, v_k]”, to delineate a trajectory on the overhead image starting from the bottom center. This conversation loop repeats until the task terminates. During executing the planned trajectory, the depth map corresponding to the overhead image is required to transform these pixel coordinates into 3D physical waypoints.

C. Navigation using Traj-VLN

If the fine-tuned VLM outputs a sequence of direction symbols (e.g., “→→→”), the agent rotates by 15 degrees in the corresponding direction for each symbol. If the model output “STOP”, the agent terminates the task and stop there.

If the model outputs “↓”, the conversation continues and it becomes the user’s turn to reply. The agent lower its head for 30 degree to capture an overhead view, which serves as the user’s response. Subsequently, during the model’s turn, it autoregressively generate a sequence of textual coordinates, which are parsed as pixel coordinates: $Traj_{pixel} =$

$\{(u_i, v_i) \mid i = 1, 2, \dots, k\}$ to delineate a trajectory on the overhead image. After executing the planned trajectory, the conversation with the VLM is restarted with updated visual observations.

Given the corresponding depth map of the overhead view, the trajectory is extended to a sequence of 3D vectors:

$$\mathbf{Traj}_{depth} = \{(u_i, v_i, d_i)^T \mid i = 1, 2, \dots, k\} \quad (1)$$

Assuming the homogeneous camera intrinsics are represented by $\mathbf{K}$ and the camera extrinsics relative to the robot base by $\mathbf{T}_{c2b}$, each vector in $\mathbf{Traj}_{depth}$ is transformed into the robot frame (where $Z_i = d_i$) as follows:

$$\mathbf{K} = \begin{pmatrix} f_u & 0 & c_u & 0 \\ 0 & f_v & c_v & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2)$$

$$(X_i, Y_i, Z_i, 1)^T = d_i \mathbf{T}_{c2b}^{-1} \mathbf{K}^{-1} (u_i, v_i, 1, \frac{1}{d_i})^T \quad (3)$$

$$\mathbf{Traj}_{3D} = \{(X_i, Y_i, Z_i)^T \mid i = 1, 2, \dots, k\} \quad (4)$$

With the 3D trajectory $\mathbf{Traj}_{3D}$ in the robot frame, the robot is controlled to iteratively move toward the closest 3D point until it is determined to have been reached. Once the current plan is deemed finished, the conversation is restarted.

D. Data Collection

Following JanusVLN [1] and StreamVLN [6], we construct our training data from the standard R2R-CE [25] (comprising approximately 10.7K trajectories and 152K samples) and RXR-CE [26] (comprising approximately 19.5K trajectories and 455.6K samples), supplemented with a subset of ScaleVLN [27] (comprising approximately 77.3K trajectories and 1.14M samples). Sampling at an action interval of 4 steps yields a total of 1.75M samples from approximately 107.5K trajectories. Each sample is represented as a tuple $(\mathbf{I}, \mathbf{O}_{his}, \mathbf{O}_{cur}, \mathbf{P})$, where $\mathbf{I}$ denote the instruction, $\mathbf{O}_{his}$ is the historical observations, $\mathbf{O}_{cur}$ is the current observation, and $\mathbf{P}$ is the supervision signal for VLMs to predict.

To generate the supervision trajectory signals, we project the future poses relative to the current pose onto the current overhead image. Let the poses of a l -length trajectory be denoted as:

$$\mathbf{Traj} = \{\mathbf{T}_1, \mathbf{T}_2, \dots, \mathbf{T}_{cur}, \dots, \mathbf{T}_l\} \quad (5)$$

where $\mathbf{T}_{cur}$ represents the pose of the current observation of a given sample:

$$\mathbf{T}_{cur} = \begin{pmatrix} \mathbf{R}_{cur} & \mathbf{t}_{cur} \\ 0 & 1 \end{pmatrix} \quad (6)$$

The future poses relative to $\mathbf{T}_{cur}$ are represented as:

$$\mathbf{Traj}_{future} = \{\mathbf{T}_i \mid cur < i \leq l\} \quad (7)$$

We project the base positions of $\mathbf{Traj}_{future}$ onto the overhead image as follows. First, we define the base pose:

$$\mathbf{T}_i^{base} = \begin{pmatrix} \mathbf{R}_i & \mathbf{t}_i - (0, 0, height)^T \\ 0 & 1 \end{pmatrix} \quad (8)$$

User: You are an autonomous navigation assistant. Your task is to follow the instruction: `<instruction>`. Based on the current image, where should you go next to follow that instruction? The closest observable point on the ground is at the bottom-center of the overhead image: (319, 479) for an image of width 640 and height 480. Do NOT output this point. Please output up to 6 future waypoint coordinates in the current overhead image as pixel coordinates, ordered from near to far. Please output the coordinates in the format like: [x1,y1] [x2,y2] ... There is an example output of pixels: [310,468], [295,440] [270,405]. Please output STOP when you have successfully completed the task. These are your historical observations in chronological order: `<history>`. Currently you can see `<image>`.

Assistant: `← ← ←`

User: You are an autonomous navigation assistant.....

Assistant: ↓

User: `<overhead image>`

Assistant: [x1,y1] [x2,y2] [x3,y3] [x4,y4] [x5,y5]

User: You are an autonomous navigation assistant.....

Assistant: ←

User: You are an autonomous navigation assistant.....

Assistant: ↓

User: `<overhead image>`

Assistant: [x1,y1] [x2,y2] [x3,y3]

User: You are an autonomous navigation assistant.....

Assistant: → → → →

User: You are an autonomous navigation assistant.....

Assistant: STOP

Fig. 3: An example of conversations during a navigation task. The placeholders “`<instruction>`”, “`<image>`”, “`<overhead image>`”, and “`<history>`” are dynamically replaced with corresponding textual and visual content to form the input sequence, which is subsequently tokenized according to the VLM’s vocabulary.

where $\mathbf{t}_i^{base} = \mathbf{t}_i - (0, 0, height)^T$ denotes the translation component. This base position is then projected onto the image plane:

$$(X_i, Y_i, Z_i, 1)^T = \mathbf{K} \mathbf{T}_{cur}^{-1} (\mathbf{t}_i^{base}, 1)^T \quad (9)$$

yielding n pixel coordinates:

$$\mathbf{Traj}_{pixel} = \{(u_i, v_i) = \frac{1}{Z_i} (X_i, Y_i) \mid i = 1, 2, \dots, n\} \quad (10)$$

Finally we filter out invisible and occluded pixels from $\mathbf{Traj}_{pixel}$ and format the remaining into a textual string “[u_1, v_1] [u_2, v_2] ... [u_k, v_k]”, which is incorporated into the chat template as the target for the VLM to predict.

E. Training of Traj-VLN

Our flagship Traj-VLN, based on Qwen3-VL-8B, was trained on a server with 8 Ascend 910C NPUs for approximately 91 hours, totally 728 NPU hours. Leveraging the 1.75M samples extracted from the three datasets, the training process comprised 16998 steps with a batch size of 128, and each step took approximately 23 seconds. The model was trained for one epoch, during which we exclusively fine-tuned the LLM and the projection layer with a learning rate of 2e-5, while keeping the vision encoder frozen. We observed that freezing the vision encoder saved

TABLE I: Comparison with SOTA methods on the VLN-CE R2R Val-Unseen and RxR Val-Unseen splits. The commonly used 2.6M training samples are derived from the standard R2R-CE (10.7K trajectories) and RxR-CE (19.5K trajectories) datasets. InternVLA-N1 additionally employs a subset of the ScaleVLN dataset, comprising 77K trajectories. Following InternVLA-N1, we extract training samples at an interval of 4 from these three datasets; however, we use a single robot height and view angles setting, yielding 1.75M samples, while InternVLA-N1 uses multiple robot height and angle configurations, resulting 4.72M samples. The training sample quantities for other methods are sourced from various additional datasets. To reduce training overhead, we freeze the vision encoder to save approximately 60% of the training duration compared to full fine-tuning.

VLM Backbone	Vision Encoder	Training Samples	Method	R2R-CE Val-Unseen				RxR Val-Unseen			
				NE↓	OS↑	SR↑	SPL↑	NE↓	SR↑	SPL↑	nDTW↑
—	—	—	Sim2Real [28] [CoRL24]	5.95	55.8	44.9	30.4	8.79	36.7	25.5	18.1
—	—	—	AO-Planner [29] [AAAI25]	5.55	59.0	47.0	33.0	7.06	43.3	30.5	50.1
—	—	—	NavMorph [30] [ICCV25]	5.75	56.9	47.9	33.2	8.85	30.8	22.8	44.2
LLaVA-Video-7B	Unfrozen	2.60M	StreamVLN [6] [arXiv25]	5.43	62.5	52.8	47.2	6.72	48.6	42.5	60.2
LLaMA-VID-7B	Frozen	3.55M	NaVid [2] [RSS24]	5.47	49.1	37.4	35.9	8.41	23.8	21.2	—
LLaMA-VID-7B	Frozen	1.84M	NaVid-4D [7] [ICRA25]	5.99	55.7	43.8	37.1	—	—	—	—
VILA-7B	Unfrozen	15.6M	NaVILA [4] [RSS25]	5.22	62.5	54.0	49.0	6.77	49.3	44.0	58.8
Qwen2.5VL-7B	Frozen	2.60M	JanusVLN [1] [ICLR26]	5.17	58.0	52.8	49.2	6.46	51.4	44.3	49.1
Qwen2.5VL-7B	Unfrozen	4.72M	InternVLA-N1 [11] [ICLR26]	4.89	60.6	55.4	52.1	6.41	49.5	41.8	62.6
Intern3VL-8B	Frozen	2.60M	Goal2Pixel [12] [arXiv26]	4.80	58.3	53.9	52.7	6.91	48.1	44.7	63.0
Qwen3VL-8B	Frozen	1.75M	Traj-VLN (ours)	4.83	65.6	58.4	51.7	6.02	52.6	43.4	62.8

approximately 62.5% of the training time. Throughout all training and evaluation runs, the total number of historical images was fixed at 8. Except for the overhead image, which was maintained at 640×480 , the remaining frames were uniformly resized to 384×384 .

For the comparative experiments, multiple models were trained under identical configurations using Qwen2.5-VL-7B as the backbone, with the LLM fine-tuned via LoRA (rank=32). To ensure a fair comparison between our method and those using pixel-goal supervision and prediction, we kept the same data, backbone and training settings across models, while varying only the supervision signals. On a server equipped with 4 Nvidia 4090 GPUs (each with 48GB of memory), each training run consumed approximately 171 hours to complete 274,266 steps.

IV. EXPERIMENTS

Our experiments are designed to investigate the following questions for evaluating Traj-VLN:

- (1) How well does Traj-VLN perform against SOTA VLN models under a comparable scale of training data?
- (2) To what extent does pixel trajectory supervision outperform pixel-goal supervision in VLN?
- (3) Can autoregressive trajectory generation serve as a chain-of-thought for improving pixel-goal prediction?
- (4) Does Traj-VLN generalize effectively to unseen real-world scenarios?

Our comparative experiments are conducted on the prominent VLN-CE benchmarks comprising 1839 R2R-CE val-unseen trajectories and 3669 RxR-CE val-unseen trajectories. We report the performance using standard VLN-CE metrics consisting of Navigation Error (NE), Oracle Success Rate (OS), Success Rate (SR) Success-weighted Path Length (SPL), and normalized Danymic Time Warping (nDTW).

A. Implementation and Experimental Setup

To compare with SOTA methods on the VLN-CE benchmark, we build our flagship model on Qwen3-VL-8B. The model is trained for one epoch on a server with 8 Ascend

910C NPUs, taking approximately 91 hours. We fully fine-tune the LLM and the projection layer with a learning rate of $2e-5$, while keeping the vision encoder frozen—a strategy that reduces approximately 62.5% of the training duration. Our training data is collected from the standard R2R-CE, RxR-CE and the commonly-used ScaleVLN dataset. Sampling at intervals of 4 steps along the trajectories, we project the future poses onto the image plane and filter out invisible and duplicate coordinates. From the 107.6K trajectories across these three datasets, we construct a total of 1.75M training samples, each consisting of an instruction, historical images, the current image, and one of the following: a sequence of pixel coordinates, several turning actions, or “STOP”.

To compare the supervision of pixel-space trajectory with pixel-goal supervision, we trained two models under identical configurations while varying only the supervision signals, denoted as Traj-VLN and InternVLA-N1-S2 shown in Table II. Each model is based on Qwen2.5-VL and fine-tuned with LoRA at a rank of 32, with the vision encoder frozen. The models are trained for one epoch on a server with 4 Nvidia 4090 GPUs, taking approximately 171 hours. The training data for these two models are largely identical, differing only in the textual sequence for VLMs to predict: a single pixel coordinate indicating the target position versus a series of pixel coordinates to delineating a trajectory. The data used for these two models are the same 1.75M samples as those used for the flagship model.

B. Results on VLN-CE Benchmark

As presented in Table I, to address the research question (1), we compare our flagship Traj-VLN with seven VLM-based methods and three top-performing traditional approaches that do not employ VLMs. NaVILA, InternVLA-N1, Goal2Pixel, and our Traj-VLN all adopt 8 uniformly sampled images as the historical observations. Among the SOTA methods with a comparable scale of training data, Traj-VLN achieves the highest SR on both evaluation splits, despite using less training data.

JanusVLN and NaVid-4D are recent methods that incor-

TABLE II: Comparison of our trajectory supervision and pixel-goal supervision on the VLN-CE R2R Val-Unseen and RxR Val-Unseen splits. In this study, we fine-tune models using identical training configurations while varying only the supervision signal. “Final-Pixel Evaluation” refers to regarding the final pixel coordinate of the predicted trajectory as the pixel-goal, therefore evaluating it using the same pipeline as “Pixel-Goal Evaluation”. “Trajectory Evaluation” refer to sequentially executing the predicted waypoints in order. During training, the Euclidean distance from the current agent position to the pixel-goal is similar to that to the final coordinate of the trajectory.

Model	Vision Encoder	Training Samples	Method (Fine-tuned by LoRA, Rank 32)	R2R-CE Val-Unseen				RxR Val-Unseen			
				NE↓	OS↑	SR↑	SPL↑	NE↓	SR↑	SPL↑	nDTW↑
Qwen2.5VL-7B	Frozen	1.75M	InternVLA-N1-System2 (Pixel-Goal Evaluation)	5.81	45.6	39.6	36.3	8.68	36.09	30.87	51.42
Qwen2.5VL-7B	Frozen	1.75M	Traj-VLN (Final-Pixel Evaluation)	5.79	49.0	43.2	40.3	7.66	41.54	35.95	56.12
Qwen2.5VL-7B	Frozen	1.75M	Traj-VLN (Trajectory Evaluation)	5.66	48.3	43.6	40.4	7.98	39.38	33.76	48.71

porate 3D information to compensate for VLMs’ inherent weakness in 3D perception. Traj-VLN surpasses JanusVLN by 5.6 and 1.2 points in SR on the two splits, respectively. Goal2Pixel and InternVLA-N1 are recent methods that adopt pixel-goal as the supervision signal. Despite using a more recent VLM backbone than InternVLA-N1, Goal2Pixel leverages less training data and keeps the vision encoder frozen. Unfreezing the vision encoder can lead to performance gains, but typically requires more training data and computational resources to mitigate overfitting. In our approach, we select Qwen3-VL as the backbone, which features an updated vision encoder, and keep it frozen during training. On the R2R-CE split, Traj-VLN achieves 58.4 in SR, improving by 3 points over InternVLA-N1. On the RxR split, Traj-VLN attains 52.6 in SR, outperforming InternVLA-N1 by 2.9 points. In summary, Traj-VLN consistently surpasses both methods that rely on different supervision signals and those that incorporate explicit 3D information to address the 3D perception challenge in VLM-based VLN-CE.

C. Comparison of Traj-VLN with Pixel-goal Supervision

For question (2), we trained two models under identical configurations while varying only the supervision signals. Since Traj-VLN fine-tunes the VLM to predict a sequence of pixel coordinates, we encounter a trade-off problem during evaluation regarding how many waypoints to execute per trajectory prediction. To study the impact of this factor, we evaluate Traj-VLN with varying numbers of waypoint executed per prediction. As the overall trends shown in the Fig. 4, executing fewer waypoints per prediction—thereby increasing the reasoning frequency—leads to performance gains in SR, albeit at the cost of lower OSR. The chart also indicates that, regardless of the number of points executed, Traj-VLN consistently outperforms pixel-goal supervision in both SR and OSR. Among all configurations, executing only one waypoint per prediction (corresponding to a movement of approximately 0.5m) yields the highest SR. Under this setting, Traj-VLN surpass pixel-goal supervision by 4 and 3.29 in SR on the R2R-CE Val-Unseen and RxR-CE Val-Unseen splits, respectively. To compare more comprehensively, we report additional metrics in Table II, where the entries under “Final-Pixel Evaluation” correspond to executing one waypoint per prediction.

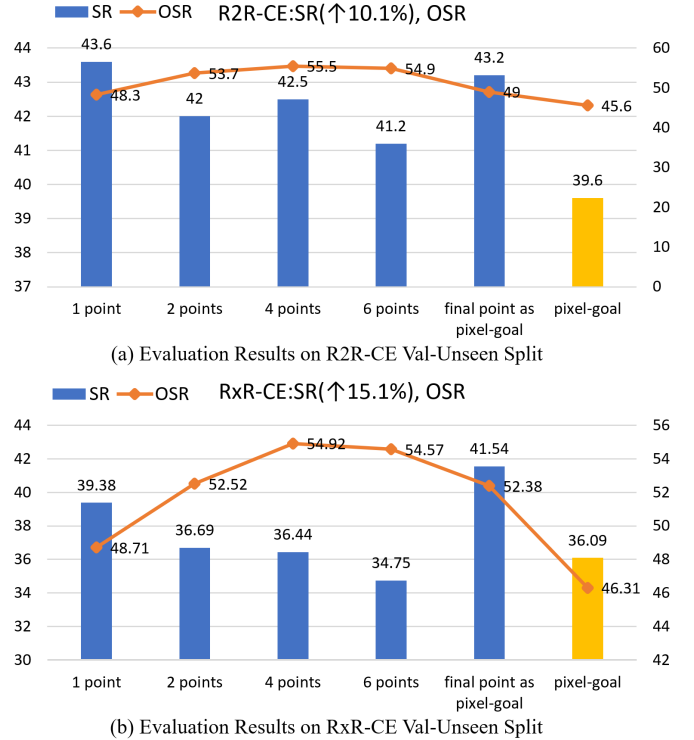


Fig. 4: Comparison of SR and OSR between pixel-goal and Traj-VLN at different numbers of execution steps per trajectory prediction. The number of pixels indicates the number of waypoints along the trajectory to execute. “final pixel as pixel-goal” refers to treating the final waypoint along the trajectory as the pixel-goal for evaluating Traj-VLN using the same evaluation scripts as pixel-goal. The results are obtained using the same models as those reported in Table II.

D. Traj-CoT Validation

Traj-VLN is built upon the hypothesis that learning spatial interaction processes over image regions yields greater VLN performance than supervising the VLMs solely with the resulting pixel-goal. Without such process-level supervision, the predicted pixel-goal may merely reflect a distribution learned from the training data, rather than a result of reasonable navigation actions. To verify the Traj-CoT mechanism, we adopt the final pixel coordinate along the trajectory predicted by Traj-VLN as the pixel goal for evaluation. The corresponding results are presented in Table II and Figure 4, where they are referred to as “Final-Pixel Evaluation” and “final point as pixel-goal” respectively. In both “Pixel-Goal

Evaluation” and “Final-Pixel Evaluation”, each conversation is terminated and restarted upon reaching the maximum of 8 movement steps. This evaluation strategy also contribute to performance improvement by increasing the number of prediction iterations.

V. CONCLUSION

In this paper, we propose Traj-VLN, the first VLN framework that enables general-purpose VLMs to learn pixel-space interactions via autoregressive trajectory generation. We divide the conversation with the VLM into two stages: the VLM outputs turning actions in the first stage and shifts to the second stage if the output is “↓”. In the second stage, the VLM is provided with an overhead image, on which it is supervised to delineate a trajectory by autoregressively outputting a series of pixel coordinates.

By learning interaction processes directly in the pixel space, Traj-VLN outperforms methods that supervise VLMs with a single pixel-goal coordinate, and achieves SOTA-level performance on the VLN-CE benchmark compared with the approaches using a similar scale of training data. Through experiments that treat the final pixel coordinate of the predicted trajectory as the pixel-goal, we further demonstrate that our trajectory supervision effectively serves as a chain-of-thought (CoT) mechanism for better pixel-goal prediction. Our qualitative results also show Traj-VLN’s capability of correct interactions with cluttered environments, which is not readily available in pretrained VLMs.

REFERENCES

- [1] Shuang Zeng, Dekang Qi, Xinyuan Chang, Feng Xiong, Shichao Xie, Xiaolong Wu, Shiyi Liang, Mu Xu, Xing Wei, and Ning Guo. “Janusvln: Decoupling semantics and spatiality with dual implicit memory for vision-language navigation.” ICLR, 2026.
- [2] Jiazhao Zhang, Kunyu Wang, Rongtao Xu, Gengze Zhou, Yicong Hong, Xiaomeng Fang, Qi Wu, Zhizheng Zhang, and He Wang. “Navid: Video-based vlm plans the next step for vision-and-language navigation.” RSS, 2024.
- [3] Meng Wei, Chenyang Wan, Jiaqi Peng, Xiqian Yu, Yuqiang Yang, Delin Feng, Wenzhe Cai et al. “Ground slow, move fast: A dual-system foundation model for generalizable vision-and-language navigation.” ICLR, 2026.
- [4] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Zaitian Gongye, Xueyan Zou, Jan Kautz, Erdem Biyik, Hongxu Yin, Sifei Liu, and Xiaolong Wang. “Navila: Legged robot vision-language-action model for navigation.” RSS, 2025.
- [5] Jiazhao Zhang, Kunyu Wang, Shaoan Wang, Minghan Li, Haoran Liu, Songlin Wei, Zhongyuan Wang, Zhizheng Zhang, and He Wang. “Uni-navid: A video-based vision-language-action model for unifying embodied navigation tasks.” arXiv, 2024.
- [6] Meng Wei, Chenyang Wan, Xiqian Yu, Tai Wang, Yuqiang Yang, Xiaohan Mao, Chenming Zhu et al. “Streamvln: Streaming vision-and-language navigation via slowfast context modeling.” arXiv, 2025.
- [7] Haoran Liu, Weikang Wan, Xiqian Yu, Minghan Li, Jiazhao Zhang, Bo Zhao, Zhibo Chen, Zhongyuan Wang, Zhizheng Zhang, and He Wang. “NaVid-4D: Unleashing Spatial Intelligence in Egocentric RGB-D Videos for Vision-and-Language Navigation.” In 2025 IEEE International Conference on Robotics and Automation (ICRA), pp. 10607-10615. IEEE, 2025.
- [8] Peizheng Li, Zhenghao Zhang, David Holtz, Hang Yu, Yutong Yang, Yuzhi Lai, Rui Song, Andreas Geiger, and Andreas Zell. “Spacedrive: Infusing spatial awareness into vlm-based autonomous driving.” CVPR, 2026.
- [9] Pingyi Chen, Yujing Lou, Shen Cao, Jinhui Guo, Lubin Fan, Yue Wu, Lin Yang, Lizhuang Ma, and Jieping Ye. “SD-VLM: Spatial Measuring and Understanding with Depth-Encoded Vision-Language Models.” NeurIPS, 2026.
- [10] Asher J. Hancock, Xindi Wu, Lihan Zha, Olga Russakovsky, and Anirudha Majumdar. “Actions as language: Fine-tuning vlms into vlms without catastrophic forgetting.” ICLR, 2025.
- [11] InternNav Team. “InternVLA-N1: An Open Dual-System Navigation Foundation Model with Learned Latent Plans.” arXiv, 2025.
- [12] Muyi Bao, Yuxin Cai, Hang Xu, Zongtai Li, Jinxi He, Jingfan Tang, Chen Lv, Ji Zhang, Yaqi Xie, and Wenshan Wang. “Goal2Pixel: Grounding Goals to Pixels for Vision-Language Navigation.” arXiv, 2026.
- [13] Shuo Wang, Yongcai Wang, Wanting Li, Xudong Cai, Yucheng Wang, Maiyue Chen, Zhizhong Su, Deying Li, and Zhaoxin Fan. “Aux-think: Exploring reasoning strategies for data-efficient vision-language navigation.” Advances in Neural Information Processing Systems 38 (2026): 30892-30916.
- [14] Duo Zheng, Shijia Huang, Lin Zhao, Yiwu Zhong, and Liwei Wang. “Towards learning a generalist model for embodied navigation.” CVPR, 2024.
- [15] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Niessner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. “Matterport3d: Learning from rgb-d data in indoor environments.” 3DV, 2017.
- [16] Anderson, Peter, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton Van Den Hengel. “Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments.” In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 3674-3683. 2018.
- [17] Alexander Ku, Peter Anderson, Roma Patel, Eugene Ie, and Jason Baldridge. “Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding.” In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 4392-4412. 2020.
- [18] Jacob Krantz, Erik Wijmans, Arjun Majumdar, Dhruv Batra, and Stefan Lee. “Beyond the nav-graph: Vision-and-language navigation in continuous environments.” In European Conference on Computer Vision, pp. 104-120. Cham: Springer International Publishing, 2020.
- [19] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub et al. “Habitat: A platform for embodied ai research.” In Proceedings of the IEEE/CVF international conference on computer vision, pp. 9339-9347. 2019.
- [20] Yicong Hong, Zun Wang, Qi Wu, and Stephen Gould. “Bridging the gap between learning in discrete and continuous environments for vision-and-language navigation.” In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 15439-15449. 2022.
- [21] Jacob krantz, Aaron Gokaslan, Dhruv Batra, Stefan Lee, and Oleksandr Maksymets. “Waypoint models for instruction-guided navigation in continuous environments.” In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 15162-15171. 2021.
- [22] Yanwei Li, Chengyao Wang, and Jiaya Jia. “Llama-vid: An image is worth 2 tokens in large language models.” In European Conference on Computer Vision, pp. 323-340. Cham: Springer Nature Switzerland, 2024.
- [23] Shaoan Wang, Yuanfei Luo, Xingyu Chen, Aocheng Luo, Dongyue Li, Chang Liu, Sheng Chen, Yangang Zhang, and Junzhi Yu. “VLingNav: Embodied Navigation with Adaptive Reasoning and Visual-Assisted Linguistic Memory.” arXiv, 2026.
- [24] Duo Zheng, Shijia Huang, Lin Zhao, Yiwu Zhong, and Liwei Wang. “Towards learning a generalist model for embodied navigation.” CVPR, 2024.
- [25] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In European Conference on Computer Vision, pages 104–120. Springer, 2020.
- [26] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 4392–4412, 2020.
- [27] Zun Wang, Jialu Li, Yicong Hong, Yi Wang, Qi Wu, Mohit Bansal, Stephen Gould, Hao Tan, and Yu Qiao. Scaling data generation in vision-and-language navigation. In ICCV, 2023.
- [28] Zihan Wang, Xiangyang Li, Jiahao Yang, Yeqi Liu, and Shuqiang Jiang. “Sim-to-real transfer via 3d feature fields for vision-and-language navigation.” arXiv, 2024.

- [29] Jiaqi Chen, Bingqian Lin, Xinmin Liu, Lin Ma, Xiaodan Liang, and Kwan-Yee K. Wong. "Affordances-oriented planning using foundation models for continuous vision-language navigation." AAAI, 2025.
- [30] Xuan Yao, Junyu Gao, and Changsheng Xu. "Navmorph: A self-evolving world model for vision-and-language navigation in continuous environments." ICCV, 2025.